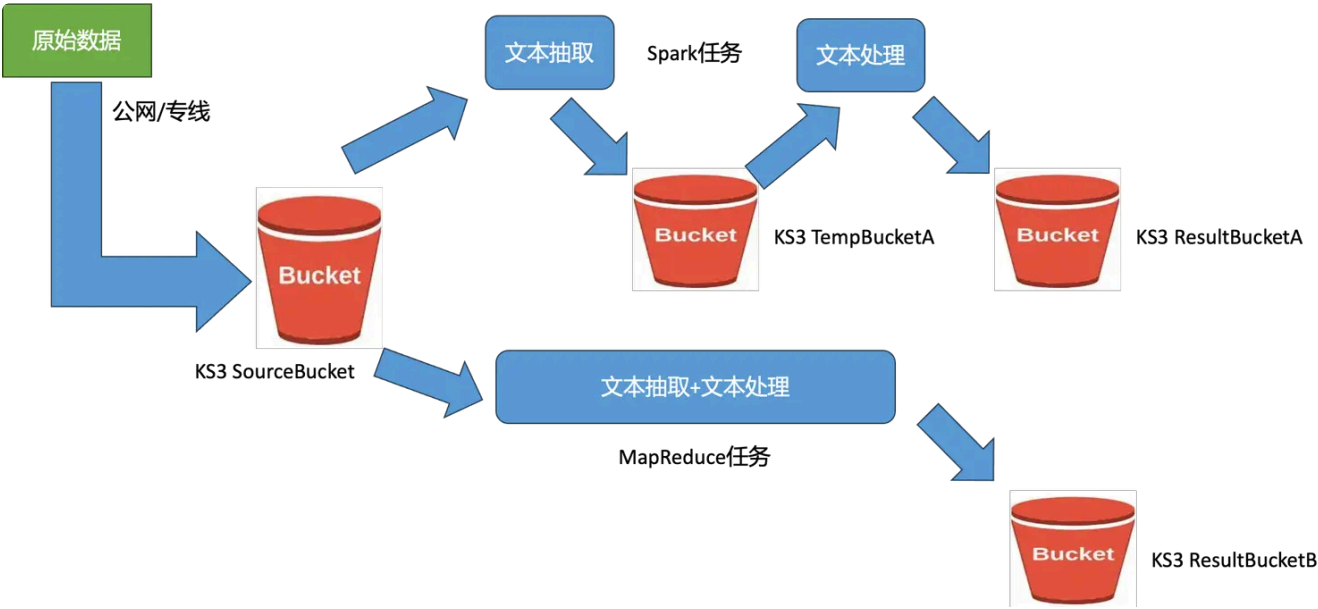


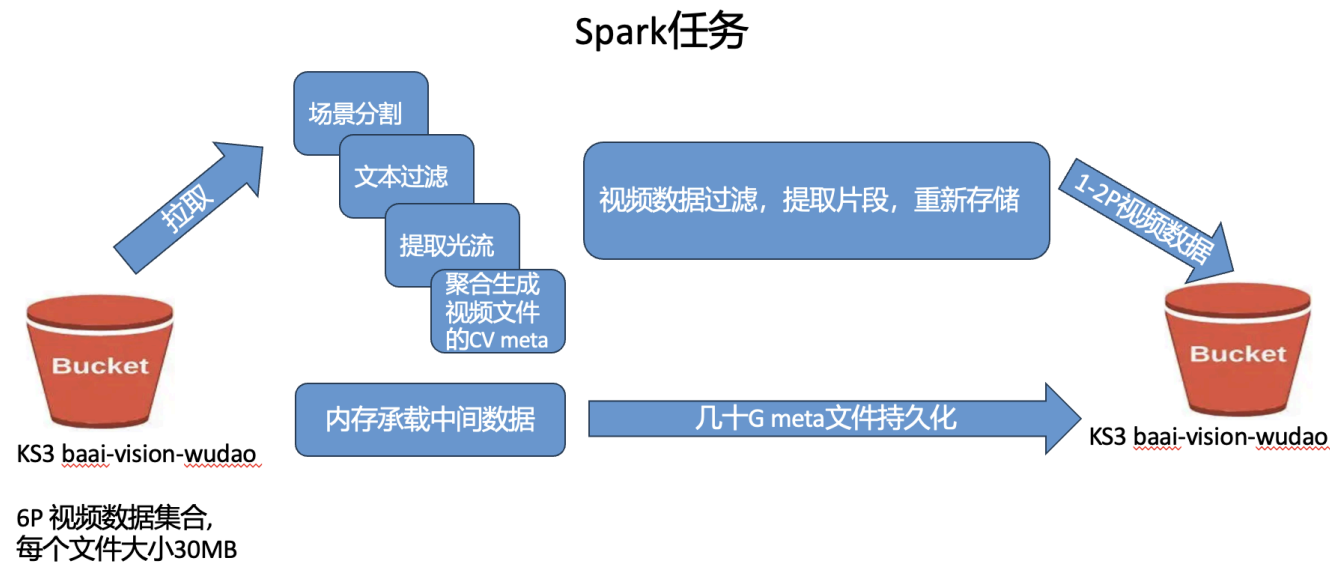
数据清洗流程

1 场景与需求分析

文本清洗



视频清洗



存算一体和存算分离的优劣势

存算一体		存算分离	
优势	劣势	优势	劣势
数据不需要在存储器和处理器之间来回传输，减少了频繁的数据搬运，大大降低了功耗和延迟可以显著提升系统的整体计算速度和能效比。	存算一体芯片不够灵活，难以适应广泛多变的计算需求，高度集成可能会增加设计和制造的复杂性，尤其是当需要更新或替换某种计算逻辑时，会带来额外的成本和技术挑战。	计算资源和存储资源可以独立升级和扩展，以满足不同工作负载的需求，实现更高的资源利用率。	由于存储和计算单元之间的物理隔离，频繁的数据交互会导致较高的延迟和较大的能耗，可能无法满足高性能计算和实时响应的要求。
建议主推存算一体方案，从 Common Crawl 下载的数据批量存放在本地，本地使用 1W 核 8258P EPC，需要 1.5PB HDFS 存储承载。		目前存算分离在使用上容易出错，不建议推荐。	

Spark 和 MapReduce 的区别

	Spark	MapReduce
计算模型	Spark 采用基于内存的计算模型，可以将中间结果存储在内存中，避免了磁盘 I/O 操作，提高了计算效率	采用基于磁盘的计算模型，需要将中间结果写入磁盘，然后再读取，增加了 I/O 操作和延迟
容错性	Spark 提供了一种基于 RDD（弹性分布式数据集）的容错机制，可以将数据划分为多个分区，并在节点故障时自动重新计算丢失的分区	采用基于数据的容错机制，需要将中间结果写入磁盘，并在节点故障时重新计算整个任务。基于 Hadoop 生态系统，提供了高度可靠的分布式数据存储（HDFS）和自动故障恢复能力，能够处理 PB 级别的数据，并轻松扩展到数千节点的集群上
编程模型	Spark 提供了一种基于 RDD 的编程模型，可以支持多种数据处理操作，如过滤、映射、聚合、连接	采用基于 Map 和 Reduce 函数的编程模型，只能支持 Map 和 Reduce 操作
实时性	Spark Streaming 支持实时数据处理，而 Spark SQL 则增强了对交互式查询的支持	MapReduce 每次操作都需要磁盘读写，两次 Map 和 Reduce 之间的数据需要落地到 HDFS，每个 Map 或 Reduce 任务都需要经历初始化、资源调度、数据本地化等一系列过程，这使得小规模或轻量级的计算任务在 MapReduce 上运行效率低下，不适应实时计算和迭代计算的需求
Spark 更适合需要高性能、迭代计算和实时分析的应用场景，而 MapReduce 则更适用于对容错性和可靠性要求较高、大规模、无需即时响应且计算逻辑相对简单的离线大数据处理任务。		

1.1 业务流程分析

TODO：考虑外网数据抓取部分（从国外 Region 通过爬虫抓取数据转国内）

文本数据的流转（Spark 流程）

网络爬虫 -> 网页数据打包

-> 通过公网数据进 SourceBucket

-> 数据清洗任务 1 ” 文本抽取 “从 SourceBucket 中读取数据 完成后 写入 TempBucket

-> 数据清洗任务 2 ” 文本处理 “从 TempBucket 中读取数据 完成后 写入 ResultBucket

-> 训练数据从 ResultBucket 读取，喂给 GPU 集群进行大模型训练

1. 网络爬虫

网络爬虫首先被部署去自动抓取互联网上的网页数据，比如文本、图片或者其他类型的数据。

2. 网页数据打包并通过公网上传至 SourceBucket

抽取的数据被打包整理后，通过公网传输至云端存储服务 KS3 中的 SourceBucket 中。这样做便于统一管理和后续分布式处理，同时确保数据的安全存储和高效访问。

3. 文本抽取

第一个数据清洗任务会从 SourceBucket 中读取原始网页数据，并对原始文本进行结构化处理，提取出真正有价值的文本信息。清洗过的文本数据不会直接覆盖原始数据，而是被写入一个临时的 TempBucket。

4. 文本处理

第二个数据清洗任务继续从 TempBucket 中读取经过初步抽取的文本数据，对数据进行更深入的数据清洗和预处理，例如文本标准化（大小写转换、拼写纠正、词干提取等）、语义分析、情感分析等高级处理，处理后，高质量、结构化的文本数据将会被写入指定的 ResultBucket 中，为下游任务（如机器学习训练）提供干净的数据。

5. 大模型训练

机器学习团队或训练平台会从 ResultBucket 中读取已经清洗和准备好的文本数据。这些数据被加载到具备强大的并行计算能力 GPU 集群中，用于训练大型神经网络模型，例如自然语言处理模型、文本生成模型、情感分析模型等。

文本数据的流转（MapReduce 流程）

1. 从 AWS Common Crawl 下载数据到 KS3:

AWS Common Crawl 提供了大规模的开放网络爬取数据集，存储在 AWS S3 中。通过跨云服务的数据迁移，借助 AWS CLI 或其他兼容工具，如 S3 客户端，将 Common Crawl 数据从 AWS S3 下载到金山云的 KS3 中。

2. 文本抽取

在下载完原始数据后，通过 MapReduce 将文件解压并抽取其中的 txt/HTML 内容，Map 阶段读取文件并解析出每条记录的文本内容，Reduce 阶段可以汇总这些文本数据。

3. 语言识别

在 MapReduce 框架下，可以编写自定义的 Map 或 Reduce 函数，对抽取的文本进行语言识别。不过，MapReduce 本身并不直接支持语言识别功能，需要结合其他工具或服务（如 Hadoop 集群内部署第三方 NLP 库）来实现这一目标。

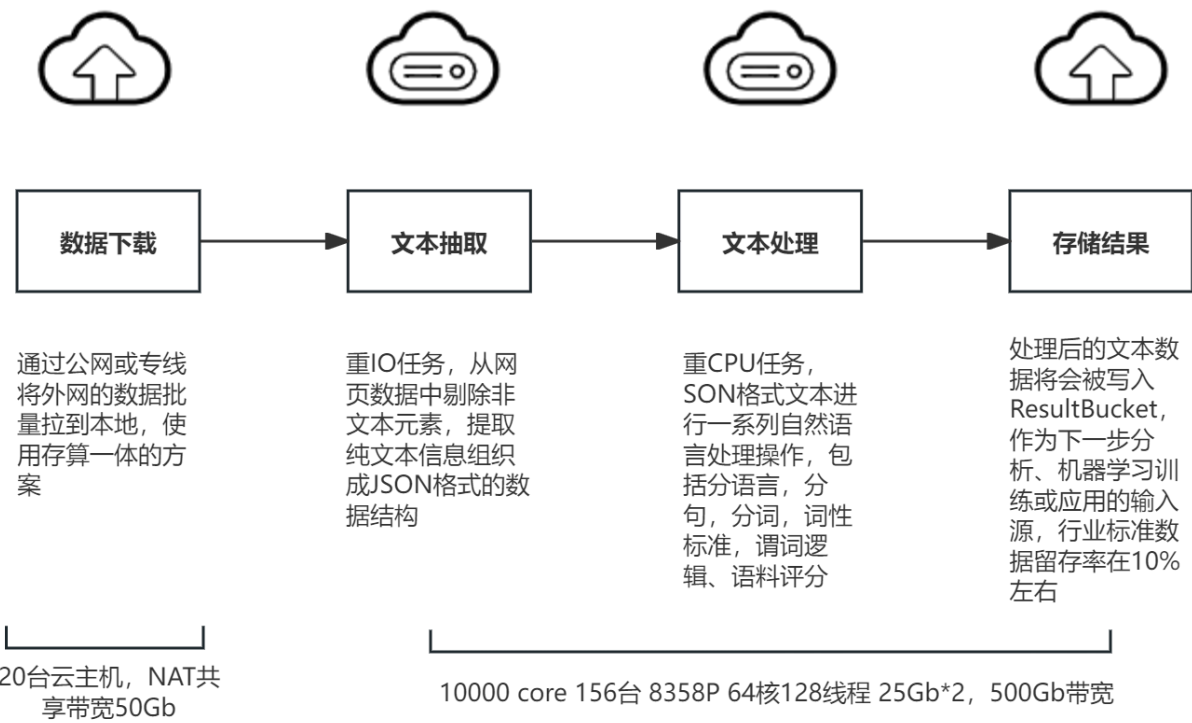
4. 文本内容打分

文本内容打分也不直接是 MapReduce 的标准功能，可以在 MapReduce 的框架下，借助 Hadoop 生态系统内的其他组件如 Spark 或 Hive 对文本进行预处理，并传递给外部服务或集群内部的算法进行打分（如基于文本的情绪分析、关键词权重分析等）。

5. 大模型训练

在 MapReduce 处理后，将清洗的数据存回 KS3 或其他存储系统，作为下一步分析、机器学习训练或应用的输入源。

文本清洗 | 存算一体方案



视频数据的流转（Spark 流程）

1. 数据拉取

从 KS3 存储桶拉取视频数据到 Spark 集群中进行处理。

2. 视频数据处理

这一阶段包括场景分割、文本过滤、光流提取和数据聚合。拉取到的数据首先会经过**场景分割**，将视频中的不同场景分离开来。然后进行**文本过滤**，去除不需要的文本信息。接下来，会对过滤后的数据进行**光流提取**，这是一种用于捕捉视频中物体运动的视觉特征的技术。提取出的光流和**其他相关信息会被聚合**在一起，生成视频文件的 CV meta。CV meta 通常包含有关视频内容的元数据，如场景描述、物体识别结果等。最后，视频数据进行过滤，提取出需要的片段，并将这些片段重新存储到存储桶中。

3. 内存处理

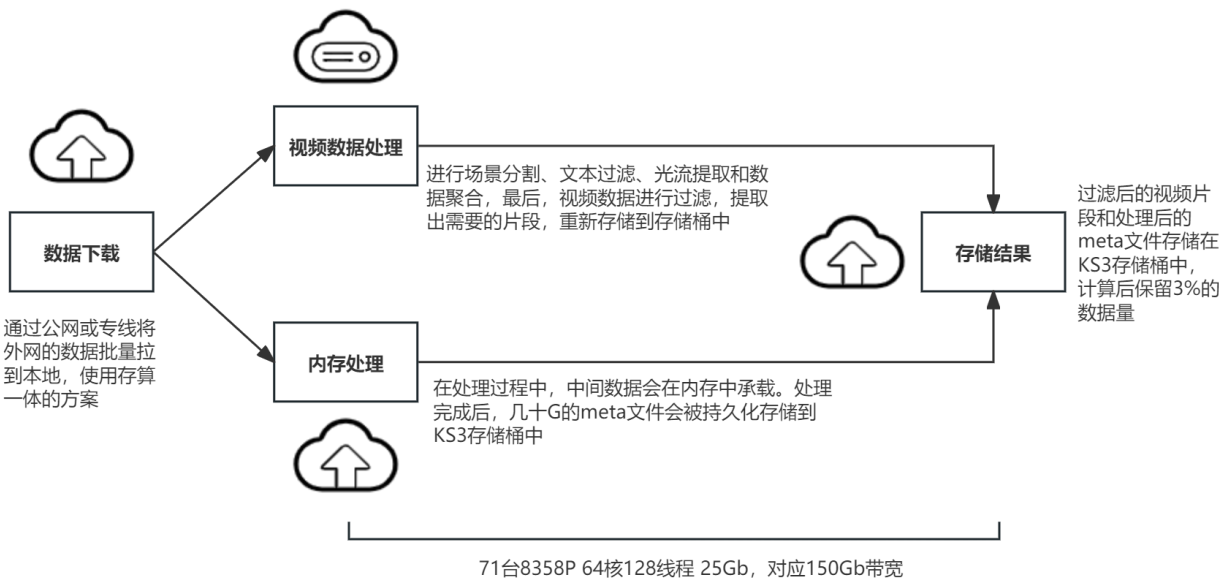
在处理过程中，中间数据会在内存中承载，以提高处理效率和减少磁盘 I/O 操作。处理完成后，几十 G 的 meta 文件会被持久化存储到 KS3 存储桶中，以便后续使用或分析。

4. 存储结果

将处理后的 meta 文件和视频片段存储回 KS3 存储桶中。

- 清洗视频资源： 11264 个并发对应 100Gb KS3

视频清洗 | 存算一体方案



数据清洗任务

1. 任务一：文本抽取（将爬取的网页转换为 JSON 格式的文本）

- 任务详述：**文本抽取是典型的 I/O 密集型任务，需要爬取的大量网页中提取出有效和有价值的内容，爬虫程序抓取网页源代码后，需解析 HTML 结构，剔除非文本元素（如 JavaScript、CSS 样式、HTML 标签等），并将提取出的纯文本信息组织成 JSON 格式的数据结构。所以当使用 CPU 进行并行处理时，要求有足够的网络带宽来支撑数据的快速读取和写入，从而避免因带宽限制而导致的任务瓶颈。

2. 任务二：文本处理（分语言，分句，分词，词性标准，谓词逻辑、语料评分）

- 任务详述：**文本抽取会针对抽取的 JSON 格式文本进行一系列自然语言处理操作，包括预处理、分词、语言识别、词性标注、命名实体识别、实体链接、关键词抽取、情感分析、主题建模、结构化信息抽取、文本摘要、语法分析、语料评分等。相对于任务一，任务二更侧重

于 CPU 密集型运算，因为自然语言的处理并不那么依赖于 I/O 速度。在这个阶段，从存储中读取数据的速度足够快，处理速度主要受限于 CPU 处理能力和内存大小。

- **清洗文本资源：清洗文本 10000 core 8358P 64core128 线程 对应 500Gb 带宽**

面壁日报

日期	任务
3.18	- 目前来看桶的限速，对于 spark 任务影响比较大，任务会超时导致失败。现在看 2 个任务都是这么失败的。目前给面壁的限速是 200gb，用户计划晚起一个 300T 的任务。我们夜间放开限速最大到 400gb。晚上和用户一起进行任务启动，来观察是否可以解决用户的问题。 - 裸金属单节点带宽达到 15Gb，触发 pgw 限流丢包，这也可能是 spark 任务失败的原因。 大数据侧优化进展： - 已将部署 gaea 服务部署到客户集群，经过客户任务验证（使用 ks3-hdfs 时该任务会卡主），有效解决任务卡主问题。该任务清洗时会加载 75 万个文件，35TB 数据。
3.19	KS3 带宽升级 200Gb-400Gb。昨晚 KS3 带宽从 200Gb 升级到 400Gb，夜间任务将带宽完全吃满，100TB 量级的任务，10 个小时进展 10%，预计 4 天可以跑完。今天下午 6 点 pgw 切换至 p4 型号，下午用户任务以计算为主已经基本上跑满 KMR EPC 的 CPU。今晚到明天凌晨，KS3 会临时提升带宽至 600Gb。我们这边会准备 10 台 EPC 搭建新的 KMR 集群，使用存算一体的方案承载部分用户数据集的清洗任务，
3.20	目前 Spark 任务第一个阶段已经跑完 100TB 数据，接下来会跑第二第三阶段（ccnet），之后会等比放大评估 8P 数据清洗需要时间明天会按照用户要求交付 12 台 EPC 搭建存算一体方案，未来计划将 MapReduce 任务切换到存算一体架构下，进一步提升清洗效率
3.21	今天 12 台 EPC 加入 KMR 集群，部署存算一体解决方案，下午用户将 MapReduce 业务切换到存算一体的集群中，计划先按照 100T 数据规模进行清洗。
3.22	目前已经跑通的核心任务： 100T Spark 任务 70T MapReduce 任务 - 目前 100TB MapReduce 任务在存算一体的集群出现 APP Master OOM 的问题。
3.23-3.24	周末帮助用户完成了存算一体集群扩容，升级之后目前已经跑通： 50TB MapReduce 任务 100TB MapReduce 任务（第一步生成 104T 数据，第二步生成 10T 数据） 接下来用户准备使用存算一体集群跑 160TB MapReduce 任务，目前帮助用户将数据从 KS3 上同步到存算一体集群之后开始跑任务。 目前来看使用存算一体集群 1500 核，跑 100TB 任务，耗时 32 小时。
3.25	- MapReduce 任务：用户 160TB MapReduce 失败，经过排查是因为 HDFS 数据分布不均匀造成的，重新 Balance 数据之后，业务恢复正常。未来计划使用 30 台满配磁盘节点替换集群中磁盘数量较少节点从而彻底解决此类问题。 - Spark 任务：用户已经跑通一个 snapshot 的整个业务流程，接下来用户开始轮训 Snapshot。
3.26	- MapReduce 任务：用户完成 160TB MapReduce 任务，用时 40 小时。今天用户运行 194T 任务，预计 60 个小时。 - Spark 任务：迭代 snapshot 未报异常

3.27	用户使用批量下载后再处理的存算一体方式继续跑任务。用户认为这种方式可行。和用户明确了KS3资源带宽上限，用户计划先下500T数据，之后本地进行处理。接下来，我们逐渐将用户存算分离的主机逐渐改配加盘加入存算一体的集群，未来数据数据都存在HDFS上。
4.1-4.2	完成存算一体集群扩容，目前用户集群91台8358P（core节点）2台6240（master节点），hdfs2副本之后，集群可用容量3.4TB。目前客户任务正常运行中。 面壁在新加坡开了6X6台云主机，作为代理访问国外一些网站，生成KB级别报文并通过公网回传，上周稳定运行，对于存储和网络上没有性能要求。
4.3	用户已经跑完500TB Mapreduce任务没有错误 今天完成32台8358p缩容，未来计划改配 高性能计算型-II-I-VIII 8358P*2（64核128线程） 1024G SATA3.5_16T*4 25G_2 * 1 高性能计算型-II-I-VII 8358P*2（64核128线程） 1024G SATA3.5_16T*12 SATA2.5_240G*2 25G_2 * 1； 计划4月8日重新加入集群。
4.8	缩容32台EPC，32台机EPC完成改配，加入用户加入存算一体集群，集群规模91台EPC 8358P

智源日报

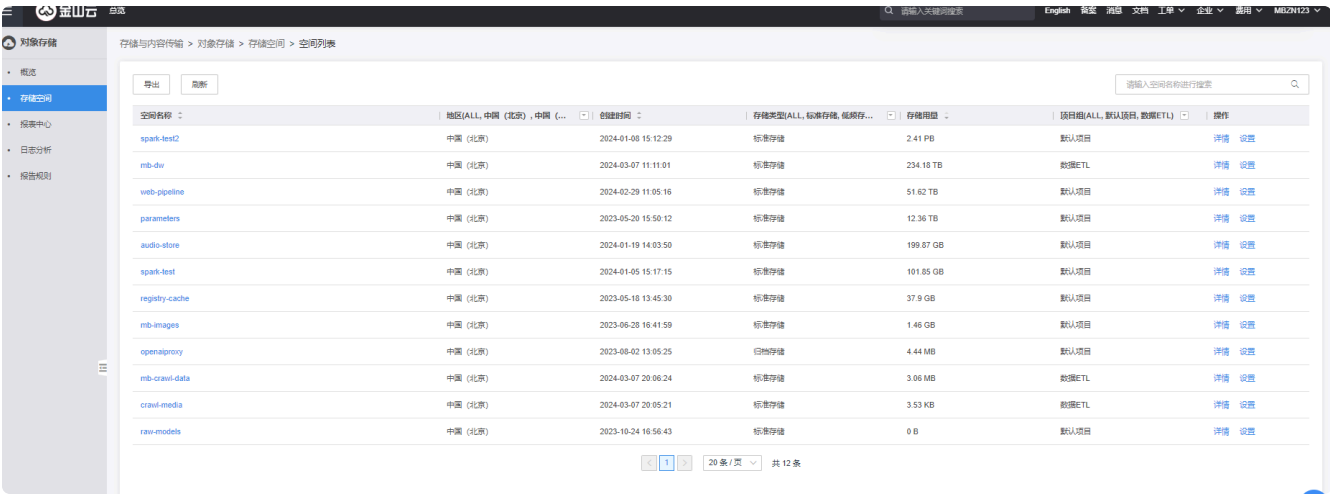
日期	项目一	项目二
3.25	上周五帮助用户同步了 resouces 和 baai-cofe 桶的 ks3-hdfs 桶的元数据，同时打开了 shadowcopy 功能 用户周末已经完成 laion-400M 数据集合清洗，用户认为清洗效率较低，经过了解用户使用方式发现用户使用 1600/1900 vcore 造成资源争抢，降低了整体清洗效率。建议用户将并发设置为物理 core。另外周末用户清洗阶段 KS3 目标桶发现大量的 head/get 请求 经过分析用户使用业务逻辑，明确在数据清洗过程中用户的目录层级较深，Spark 框架会产生大量的临时目录的读写遍历操作，导致 head/get 增加，目前该现象并没有对用户的业务产生影响，周日研发提升了 head 的 QPS。	- 用户遇到了 KMR 集群权限问题，用户错误执行命令导致集群登录不上去。今天帮助用户重建集群，问题解决 - 今天用户计划跑一个任务，已经指导用户同步桶 baai-vison-wudao ks3-hdfs 元数据，根据物理 core 设定并发数量，目前任务运行一切正常。 资源： KS3 带宽目前给 BAAI 限速 100G，未来项目一，二并行可能会是瓶颈
3.26	程序目前运行正常	用户想要加速集群处理速度，预计扩容 60 台服务器。 方案验证：客户倾向于使用多机并行下载，并行处理，python 多进程方式，单机处理提升处理效率，同时降低失败试错的成本。今天下午帮助用户下载文件到本地，用户在本地起任务
3.27	目前项目运行平稳	客户已扩容 34 台 8358p
3.28-4.1	项目运行平稳	问题节点 145 修复完成，目前用户使用过程中没有发现错误；问题节点 120 无法修复，之后会被替换。 预计明天用户扩容 28 台 8358P 集群规模达到 70 台裸金属。
4.2	目前项目运行平稳	集群新扩容 27 台 8358P，目前集群规模 71 台 8358P（core 节点）2 台 6240（master 节点），目前用户项目运行平稳。
4.3	目前项目运行平稳	今天将扩容的 27 台 8358P 的 4 块 NVMe SSD 做成了 LVM 方便用户使用，目前用户项目平稳运行。

4.7	目前项目运行平稳	节前用户集群集群加入 27 台裸金属，每台裸金属运行 64*4 个并发一共增加 6400 个并发，总计 17000 个并发下载数据。昨晚高并发触发桶级别限速，现象是：用户端任务大量失败，超时后客户端主动断开链接，服务器段出现大量 429 报错。之后把带宽限速调整到 150Gb，用户反馈使用正常。未来用户峰值并发不会超过 17000。 用户反馈目前已经清洗了 1PB 数据。
4.8	目前项目运行平稳 30 台 6240 32 核 64 线程	用户目前使用 71 台 EPC 处理任务，下载使用 17000 个并发，KS3 带宽 150Gb，目前用户集群运行平稳。
4.10	目前项目运行平稳	今天中午 12 点用户大并发上传 KS3 数据引起大量服务端 429，客户端长传失败。指导用户降低并发，问题解决

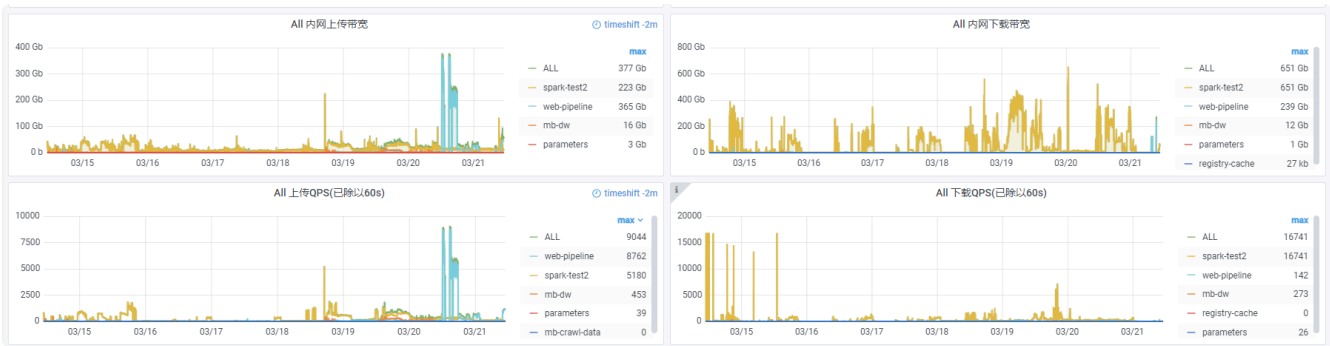
问题：项目一和项目二是共同用了 71 台 EPC 吗？

1.2 技术需求分析

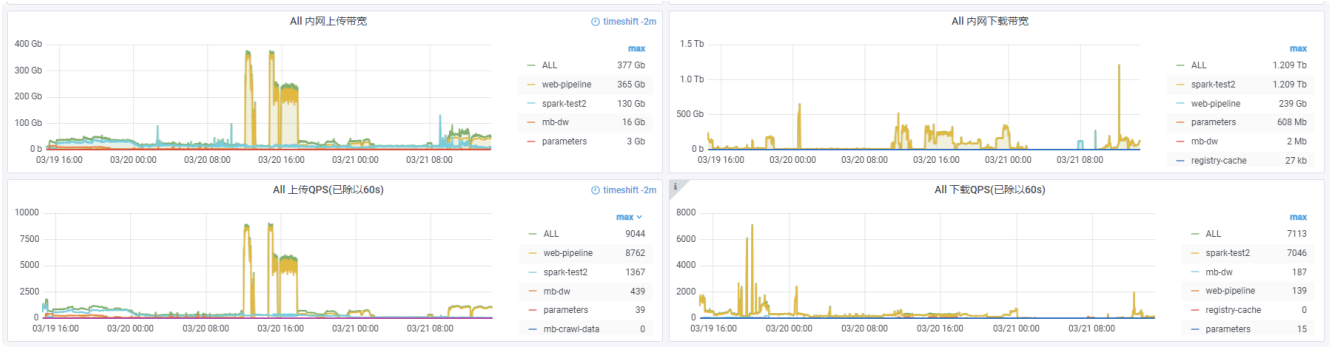
XX 项目训练数据集 Bucket 容量



XX 项目训练数据集 Bucket 吞吐



7天



48小时

大数据计算平台

弹性计算

高性能对象存储

2 基于 存算分离 / 存算一体 + 标准对象存储 解决方案

(中短期方案，24H1)

【存算一体方案】

- CPU 6240 18core*2 内存：512GB，硬盘 10*16TB
- 300TB 数据 ~ 36Core ~ 512GB
- **结论：100TB 数据 ~ 12Core ~ 128GB**

3 基于 存算分离 + 全闪对象存储 解决方案

(中长期方案，24H2，依赖全闪对象存储)

基于全闪对象存储 PB 级容量能够提供 Tb 级吞吐的超高性能，所有数据清理任务能够全部采用存算分离的架构执行。整个数据清理讲具备充分的计算和存储弹性，为客户提供更友好的体验和更高性价比。

【存算分离方案】

- 参考 面壁项目：400TB 数据 ~ 80 台物理机（32 核） ~ 400Gb~600Gb 带宽
- 参考 小红书项目：2~3PB 容量 对应 4Tb 带宽 计算资源未知
- **结论（仅文本抽取任务）：CPU/ 带宽配比： 10000core 对应 600Gb~1000Gb 带宽**

实例：面壁 100TBSpark 任务，吃满 7000 核，需要 400Gb~600Gb 带宽，完成时间 16 小时

待定问题：如果对于整个数据清理流程（任务一 + 任务二 + 任务三）

4 时间安排

- 2024.03 ~ 2024.06: 主推 存算一体 / 存算分离 + 标准对象存储 解决方案
- 2024.XX.XX：全闪对象存储公测（KS2.6.0 版本上线）
- 2024.XX.XX：全闪对象存储线上回归结果 按照读写 1：1 计算，达到 PL3 水平（单 PB 容量 兑付 1Tb 吞吐、读写比例 1：1）
- 2024.XX.XX：基于全闪对象存储的清洗业务模拟测试：100TB 任务，100Gb 吞吐，1500 核，耗时预计 ~60 小时
- 2024.XX.XX：全闪对象存储线上交付能力（四周内可交付）达到 1PB（基于面壁项目推算 典型客户肯可能运行任务所使用的容量），全链路达到 PL3 水平以上，最好 2 倍 PL3v2
- 2024.XX.XX：默认推荐方案改为 存算分离（盖亚） + 全闪对象存储方案